

Guia de Referencia do Sistema

Objetivo

Este projeto implementa um servico fake de streaming de musicas para comparar tecnologias de invocacao remota em Computacao Distribuida. O sistema nao transmite arquivos MP3. Ele trabalha com dados de catalogo: usuarios, musicas e playlists.

O mesmo dominio foi implementado em 4 tecnologias e 2 linguagens:

Linguagem	REST	gRPC	GraphQL	SOAP
Python	8001	8002	8003	8004
Java	8101	8102	8103	8104

Modelo de Dados

O sistema usa persistencia em memoria. Sempre que um servico sobe, ele inicia com a mesma massa de dados:

- 250 usuarios
- 500 musicas
- 400 playlists

Entidades:

- User: id, name, age
- Music: id, name, artist
- Playlist: id, name, userId, musicIds

Relacionamentos:

- Uma playlist pertence a um usuario.
- Uma playlist contem varias musicas.
- Uma musica pode aparecer em varias playlists.

Estrutura do Projeto

Pastas principais:

- python/rest: REST em Python com FastAPI
- python/grpc: gRPC em Python com grpcio
- python/graphql: GraphQL em Python com Strawberry
- python/soap: SOAP em Python com Spyne
- java/rest: REST em Java
- java/grpc: gRPC em Java
- java/graphql: GraphQL em Java
- java/soap: SOAP em Java
- shared/proto: contrato Protobuf usado no gRPC
- load-tests: testes de carga com Locust
- report: relatorios e graficos
- scripts: comandos prontos para setup, execucao e demonstracao

Preparacao do Ambiente

Execute uma vez antes da demonstracao:

```
./scripts/setup-python.ps1
```

Para Java gRPC, tambem execute:

```
./scripts/setup-maven.ps1
```

Observacao: Java REST, Java GraphQL e Java SOAP usam apenas a JDK. Java gRPC usa Maven local porque precisa gerar codigo a partir do arquivo .proto.

Como Subir os Servicos

Abra um terminal separado para cada servico que quiser demonstrar.

Python:

```
./scripts/run-python-rest.ps1
./scripts/run-python-grpc.ps1
./scripts/run-python-graphql.ps1
./scripts/run-python-soap.ps1
```

Java:

```
./scripts/run-java-rest.ps1
./scripts/run-java-grpc.ps1
./scripts/run-java-graphql.ps1
./scripts/run-java-soap.ps1
```

Para a apresentacao ao professor, a forma mais simples e mostrar primeiro o REST Python ou REST Java, porque o CRUD fica facil de visualizar via curl/Postman.

CRUD Completo com REST

Os exemplos abaixo usam REST Python na porta 8001.

Se estiver usando REST Java, troque a porta para 8101.

1. Listar Usuarios

```
curl.exe http://127.0.0.1:8001/users
```

2. Criar Usuario

```
curl.exe -X POST http://127.0.0.1:8001/users `
-H "Content-Type: application/json" `
-d "{\"name\":\"Joao Silva\",\"age\":25}"
```

O sistema deve retornar um usuario criado, normalmente com id 251.

3. Consultar Usuario por ID

```
curl.exe http://127.0.0.1:8001/users/251
```

4. Atualizar Usuario

```
curl.exe -X PUT http://127.0.0.1:8001/users/251 `
-H "Content-Type: application/json" `
-d "{\"name\":\"Joao Atualizado\",\"age\":26}"
```

5. Remover Usuario

```
curl.exe -X DELETE http://127.0.0.1:8001/users/251
```

6. Confirmar Remocao

```
curl.exe http://127.0.0.1:8001/users/251
```

Depois da remocao, o servico deve retornar erro de recurso nao encontrado.

CRUD de Musicas

Criar Musica

```
curl.exe -X POST http://127.0.0.1:8001/musics `
-H "Content-Type: application/json" `
-d "{\"name\":\"Musica Nova\",\"artist\":\"Artista Novo\"}"
```

Consultar Musica

```
curl.exe http://127.0.0.1:8001/musics/501
```

Atualizar Musica

```
curl.exe -X PUT http://127.0.0.1:8001/musics/501 `
-H "Content-Type: application/json" `
-d "{\"name\":\"Musica Atualizada\",\"artist\":\"Artista Atualizado\"}"
```

Remover Musica

```
curl.exe -X DELETE http://127.0.0.1:8001/musics/501
```

CRUD de Playlists

Criar Playlist

```
curl.exe -X POST http://127.0.0.1:8001/playlists `
-H "Content-Type: application/json" `
-d "{\"name\":\"Minha Playlist\",\"userId\":1,\"musicIds\":[1,2,3,4,5]}"
```

Consultar Playlist

```
curl.exe http://127.0.0.1:8001/playlists/401
```

Atualizar Playlist

```
curl.exe -X PUT http://127.0.0.1:8001/playlists/401 `
-H "Content-Type: application/json" `
-d "{\"name\":\"Playlist Atualizada\",\"userId\":1,\"musicIds\":[10,11,12]}"
```

Remover Playlist

```
curl.exe -X DELETE http://127.0.0.1:8001/playlists/401
```

Consultas Relacionais dos Slides

Listar playlists de um usuario:

```
curl.exe http://127.0.0.1:8001/users/1/playlists
```

Listar musicas de uma playlist:

```
curl.exe http://127.0.0.1:8001/playlists/1/musics
```

Listar playlists que contem uma musica:

```
curl.exe http://127.0.0.1:8001/musics/1/playlists
```

Demonstracoes Prontas

REST:

```
./scripts/demo-rest.ps1 -BaseUrl http://127.0.0.1:8001
./scripts/demo-rest.ps1 -BaseUrl http://127.0.0.1:8101
```

GraphQL:

```
./scripts/demo-graphql.ps1 -BaseUrl http://127.0.0.1:8003/graphql
./scripts/demo-graphql.ps1 -BaseUrl http://127.0.0.1:8103/graphql
```

SOAP:

```
./scripts/demo-soap.ps1 -BaseUrl http://127.0.0.1:8004
./scripts/demo-soap.ps1 -BaseUrl http://127.0.0.1:8104
```

gRPC Python:

```
$env:PYTHONPATH = "$(Get-Location);$(Join-Path (Get-Location) 'python/grpc/generated')"
./venv/Scripts/python.exe -m python.grpc.client_demo
```

gRPC Java:

```
$env:GRPC_TARGET = "127.0.0.1:8102"
$env:PYTHONPATH = "$(Get-Location);$(Join-Path (Get-Location) 'python/grpc/generated')"
./venv/Scripts/python.exe -m python.grpc.client_demo
```

Como Mostrar ao Professor

Roteiro recomendado:

1. Explique que o sistema é fake e trabalha com metadados.
2. Mostre o modelo: usuarios, musicas e playlists.
3. Suba o REST Python.
4. Execute create, read, update e delete de usuario.
5. Mostre as consultas relacionais dos slides.
6. Suba REST Java e mostre que o mesmo contrato funciona em outra linguagem.
7. Mostre rapidamente GraphQL, SOAP ou gRPC para provar as outras tecnologias.
8. Mostre que os testes de carga estão configurados com Locust.

Frase útil:

"Professor, todas as implementações usam a mesma massa inicial em memória. Assim a comparação fica focada na tecnologia de comunicação, não nos dados."

Testes de Carga

O projeto usa Locust.

Carga moderada:

```
./scripts/run-load.ps1 -Class MusicHttpUser -HostUrl http://127.0.0.1:8001 -Users 100 -SpawnRate 20 -RunTime 2m -Protocol rest -Out python_rest_moderada
```

Carga alta:

```
./scripts/run-load.ps1 -Class MusicHttpUser -HostUrl http://127.0.0.1:8001 -Users 400 -SpawnRate 80 -RunTime 2m -Protocol rest -Out python_rest_alta
```

Para GraphQL, altere o protocolo e a porta:

```
./scripts/run-load.ps1 -Class MusicHttpUser -HostUrl http://127.0.0.1:8003 -Users 100 -SpawnRate 20 -RunTime 2m -Protocol graphql -Out python_graphql_moderada
```

Para SOAP:

```
./scripts/run-load.ps1 -Class MusicHttpUser -HostUrl http://127.0.0.1:8004 -Users 100 -SpawnRate 20 -RunTime 2m -Protocol soap -Out python_soap_moderada
```

Para gRPC:

```
./scripts/run-load.ps1 -Class MusicGrpcUser -HostUrl 127.0.0.1:8002 -Users 100 -SpawnRate 20 -RunTime 2m -Protocol grpc -Out python_grpc_moderada
```

Depois dos testes, gere os graficos:

```
./venv/Scripts/python.exe ./report/generate_charts.py
```

Os arquivos ficam em report/results.

Observacoes Importantes

- Não é necessário Docker para apresentar.
- Não há banco de dados externo; os dados ficam em memória.
- Ao reiniciar o serviço, a massa inicial volta ao estado original.
- Para mostrar CRUD ao vivo, REST é a melhor opção por ser mais visual.
- O arquivo docs/examples.http pode ser usado em extensões REST Client ou adaptado para Postman.

Checklist da Apresentação

- Rodar setup Python.
- Subir REST Python.
- Criar, consultar, atualizar e remover usuário.
- Criar, consultar, atualizar e remover música.
- Criar, consultar, atualizar e remover playlist.
- Mostrar playlists de usuário.
- Mostrar músicas de playlist.
- Mostrar playlists que contêm música.
- Mostrar uma segunda linguagem, preferencialmente Java REST.
- Explicar que as demais tecnologias seguem o mesmo domínio.
- Rodar ou mostrar comando de Locust para carga moderada e alta.